



GSoC 2026 Application

Open Responses Protocol Support &
Generative UI Rendering Engine
for API Dash

Shridhar Panigrahi

sridharpanigrahi2006@gmail.com

github.com/sridhar-panigrahi

linkedin.com/in/shridhar-panigrahi-8bb176362

Polaris School of Technology, Bangalore

B.Tech Computer Science (AI & ML) | 1st Year | Expected 2029

IST (UTC+5:30)

Name and Contact Information

Name: Shridhar Panigrahi
Email: sridharpanigrahi2006@gmail.com
GitHub: [sridhar-panigrahi](https://github.com/sridhar-panigrahi)
LinkedIn: [shridhar-panigrahi-8bb176362](https://www.linkedin.com/in/shridhar-panigrahi-8bb176362)
Time zone: IST (UTC+5:30)
University: Polaris School of Technology, Bangalore
Program: B.Tech CSE (AI & ML), 1st Year
Graduation: Expected 2029

Motivation & Past Experience

1. Have you worked on or contributed to a FOSS project before?

Yes. I have **11 merged PRs** across **5 open-source projects**. API Dash is the project I've been focused on for GSoC, but I've also contributed to large codebases in Python, Java, Go, and Rust:

- **astropy** – 3 merged PRs ([#19403](#), [#19404](#), [#19450](#)): Fixed an f-string bug, silent data corruption in `FITS_rec`, and table index corruption on failed row assignment. **500k+** line Python codebase.
- **Hyperledger Besu** – 4 merged PRs ([#10020](#), [#10042](#), [#10051](#), [#10060](#)): Fixed a missing return in transaction pool, wrong hash in `eth_estimateGas`, a TOCTOU race, and unsigned underflow. Java/Ethereum client.
- **Hyperledger Fabric** – 3 merged PRs ([#5422](#), [#5424](#), [#5430](#)): Fixed gossip protocol error handling and iterator/resource leaks. Go codebase.
- **LFDT-Lockness/generic-ec** – 1 merged PR ([#59](#)): Added secp384r1 (NIST P-384) curve support – first non-32-byte scalar curve. Rust, 180 tests passing.

API Dash contributions:

- [PR #1279](#) – Added input validation for empty API key and endpoint URL fields in the AI Model Selector dialog (fixes [#1183](#)). This was my first contribution and helped me understand the Riverpod state management layer.
- [PR #1290](#) – Added support for custom OpenAI-compatible LLM providers (implements [#1175](#)). This allows users to connect API Dash to services like Groq, OpenRouter, Mistral. Building this gave me a thorough understanding of the `genai` package internals.
- [Idea Doc PR #1321](#) – Submitted a detailed initial idea document for [Idea 5: Open Responses & Generative UI](#), including architecture overview, code examples, PoC screenshots, and video demos.
- [PoC PR #1358](#) – Built a working proof-of-concept for Open Responses parsing and A2UI rendering (7 new files, 11 modified files, ~1,100 lines of code) with video demonstrations: [Open Responses Viewer demo](#) and [A2UI Renderer demo](#).

2. What is your one project/achievement that you are most proud of? Why?

Honestly, the PoC I built for this project. I spent a couple of weeks just reading through the API Dash codebase before I wrote a single line – understanding how `ResponseBodyView` controls rendering, how `ModelProvider` abstracts different AI APIs, how `Previewer` decides what widget to show. Then I built the whole thing end-to-end: sealed classes for the data models, auto-detection

logic, structured viewer widgets, A2UI component renderer, all wired into the existing response pipeline.

The part that took the longest wasn't writing the widgets – it was figuring out where each piece should live. Should the Open Responses models go in `packages/genai` or in the main app? (Answer: `genai`, because other packages might need them.) Should A2UI detection happen in `ResponseBody` or `ResponseBodySuccess`? (Answer: `ResponseBody._resolveViewOptions()`, because that's where all format detection already lives.) Getting these decisions right required actually understanding the architecture, not just grepping for keywords.

3. What kind of problems or challenges motivate you the most?

I'm drawn to problems where there's a clear gap between what tools currently do and what they should do. The gap I spotted here – every API testing tool showing AI responses as raw JSON walls – is exactly the kind of thing that motivates me. Millions of developers test AI APIs daily, and nobody has built a clean way to visualize structured AI outputs in a general-purpose API client.

I enjoy working on things where understanding the spec matters. Reading through the [Open Responses specification](#), the [A2UI component model](#), and mapping them to Flutter's widget system was genuinely interesting. These are emerging standards, and building tooling around them early means shaping how developers will interact with them.

4. Will you be working on GSoC full-time?

I will be working on GSoC full-time during the summer. My university's summer break aligns with the GSoC coding period, so I have no classes or exams during that time. I can comfortably commit 15–20 hours per week throughout the project, and more during weeks with no other commitments.

5. Do you mind regularly syncing up with the project mentors?

Not at all – I actively welcome it. I've already been engaging in the [GSoC weekly calls](#) and the [Discussion threads for Idea 5](#). Regular sync-ups help me stay aligned with what the mentors expect and catch design mismatches early rather than after a week of coding in the wrong direction. I'm available on email and video calls.

6. What interests you the most about API Dash?

Two things stand out. First, API Dash already handles an impressive range of response types – JSON, images, audio, video, PDF, CSV, SSE – and it does so through a clean, extensible rendering pipeline. That makes it realistic to add AI-specific rendering without building everything from scratch.

Second, the existing GenAI infrastructure is thoughtfully designed. The `ModelProvider` pattern, the way `outputFormatter` abstracts provider-specific parsing, and the Stac-based SDUI pipeline for AI-generated UIs show that the project already has a vision for AI-native features. What I want to build is a natural extension of that vision.

7. Can you mention some areas where the project can be improved?

- **AI response visualization:** Right now, AI API responses are displayed as raw JSON. Adding structured visualization for AI-specific response formats would be a natural and high-impact improvement.
- **Streaming response experience:** Modern AI responses include typed events (tool calls, text deltas, reasoning updates). A stream-aware viewer would make debugging agentic workflows much easier.
- **Multi-turn conversation debugging:** When building agents that chain multiple API calls together via `previous_response_id`, there's no way to see the full conversation flow. A timeline view would be very helpful.
- **Agent UI preview:** With specs like [A2UI](#) and [Flutter's GenUI SDK](#), agents can describe

interactive UIs. Being able to preview them directly in the API client would be a unique differentiator.

8. Have you interacted and helped the API Dash community?

- **GitHub Discussions:** Participated in the [Idea 5 discussion \(#1227\)](#), shared my approach, and engaged with other contributors and mentors.
- **PRs and Reviews:** Submitted [PR #1279](#), [PR #1290](#), [Idea Doc PR #1321](#), and the [PoC implementation \(PR #1358\)](#).
- **Weekly Calls:** Attending the GSoC weekly sync-up calls on [Luma](#).
- **Discord:** Active in the [#gsoc-foss-apidash](#) channel, discussing design decisions and helping answer questions.

Project Proposal Information

1. Proposal Title

Open Responses Protocol Support & Generative UI Rendering Engine for API Dash

2. Abstract

Every API testing tool shows AI responses as raw JSON walls. I built a fix.

See it working now: [Open Responses Viewer demo](#) · [A2UI Renderer demo](#)

Postman, Insomnia, Thunder Client – every tool I’ve tried renders an agentic response (reasoning + tool calls + results + message) as one flat JSON blob. You scroll, you squint, you manually join `call_id` values in your head. **Millions of developers** hit this every day. Nobody has fixed it in a general-purpose API client yet.

I want to fix it in API Dash – making it the **first cross-platform API client** with native support for the **Open Responses protocol** and **A2UI component rendering**. When API Dash detects an Open Responses response, it will:

- **Auto-switch to a structured view** – reasoning traces become collapsible cards, tool calls get matched to their results via `call_id`, messages render with markdown and inline images.
- **Stream progressively** – each output item appears as a card the moment the SSE event fires, text fills in as deltas arrive.
- **Chain conversations** – multi-turn agentic loops linked via `previous_response_id` show up as a connected timeline instead of isolated requests.
- **Render A2UI components** – when an agent returns UI descriptors (forms, tables, dashboards), API Dash builds real interactive Flutter widgets from them.

I already built a working **proof-of-concept** (~1,100 lines, 7 new files, 11 modified) before writing this proposal – because I wanted to know if the approach actually works, not just claim it would. The GSoC project extends this PoC to production quality: streaming, conversation chaining, DashBot integration, and a full test suite.

3. Detailed Description

The Problem (and Why It Matters)

I’ll be concrete about the problem. When you test an AI API that uses tool calling – say, asking a model to look up the weather – the response you get back looks something like this:

```
{
  "id": "resp_abc123",
  "object": "response",
  "status": "completed",
  "output": [
    {
      "type": "reasoning",
      "id": "rs_001",
      "summary": [{"type": "summary_text",
        "text": "Deciding to call weather tool..."}]
    },
    {
      "type": "function_call",
      "id": "fc_001",
      "name": "get_weather",
```

```

    "call_id": "call_xyz",
    "arguments": "{\"city\": \"Tokyo\"}"
  },
  {
    "type": "function_call_output",
    "call_id": "call_xyz",
    "output": "{\"temp\": 22}"
  },
  {
    "type": "message",
    "id": "msg_001",
    "content": [{"type": "output_text",
      "text": "It's 22C in Tokyo."}]
  }
],
"usage": {"input_tokens": 150,
  "output_tokens": 45, "total_tokens": 195}
}

```

In every API testing tool today, that renders as a raw JSON tree. Here is what that experience looks like compared to what this project delivers:

TODAY

Postman • Insomnia • Thunder Client

Raw JSON tree. One flat output[] array.

- Which `function_call_output` matches which call? Compare `call_ids` manually.
- Where is the actual answer? Scroll past reasoning blocks to find it.
- What order did things happen? Reconstruct from a flat array.
- SSE stream? Raw event text dumped in the body pane.
- A2UI components? Unrendered JSON blobs.

AFTER

API Dash with this project

Structured card view. Each output item is its own widget.

- Tool calls & results auto-grouped by `call_id` — matched in one expandable card.
- Message rendered as formatted markdown, inline images included.
- Reasoning collapsed by default, full chain-of-thought one click away.
- SSE stream? Cards appear live, text fills in character by character.
- A2UI components? Real interactive Flutter widgets — tap, type, interact.

You have to mentally track which `function_call_output` matches which `function_call` by comparing `call_id` values, scroll past reasoning blocks to the actual message, and somehow figure out the execution flow from a flat array.

The [Open Responses protocol](#) is becoming a cross-provider standard. [Google's A2UI specification](#) adds another dimension: agents can return structured UI component descriptors in their responses, describing interactive interfaces (forms, dashboards, data tables) that a client should render. There is currently no way to preview what those rendered UIs would look like without building a full client application.

API Dash is well-positioned to solve both problems. It already has a powerful response rendering

pipeline (Previewer handles 60+ MIME types), a GenAI provider architecture in `packages/genai`, and a Stac-based SDUI pipeline for AI-generated UIs. The gap is connecting these newer AI response formats to that existing infrastructure.

What I've Already Built (Proof of Concept)

I didn't want to just write about what I'd do – I went ahead and built a working prototype first. The PoC (PR #1358) is built and open for review. You can see it in action here: [Open Responses Viewer demo](#) · [A2UI Renderer demo](#).

PoC Submitted — PR #1358 Open for Review

7 new files · 11 modified files · ~1,100 lines of Dart code

Here's what's implemented and how each piece works:

1. Open Responses Data Models (`packages/genai/lib/models/open_responses.dart` – 269 lines)

Typed Dart sealed classes for the full [Open Responses](#) output schema:

```
sealed class OutputItem {
  const OutputItem();
  String get id;
  String get type;
  String get status;

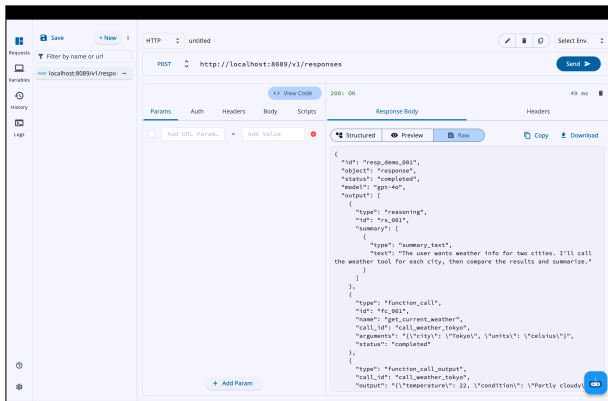
  factory OutputItem.fromJson(Map<String, dynamic> json) {
    return switch (json['type'] as String? ?? '') {
      'message' => MessageOutputItem.fromJson(json),
      'function_call' => FunctionCallOutputItem.fromJson(json),
      'function_call_output' =>
        FunctionCallResultItem.fromJson(json),
      'reasoning' => ReasoningOutputItem.fromJson(json),
      _ => UnknownOutputItem.fromJson(json),
    };
  }
}
```

2. Structured Response Viewer (`lib/widgets/open_responses_viewer.dart` – 537 lines)

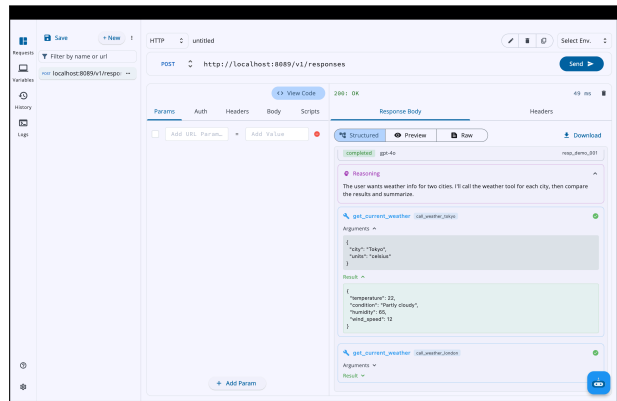
A full widget tree that renders each output item type as a distinct, themed card:

- `_MessageCard` – Role-labeled chat bubbles with GitHub-flavored markdown rendering via `MarkdownBody`, user messages right-aligned with avatar, assistant messages left-aligned.
- `_ReasoningCard` – Collapsible reasoning sections showing summary by default, expandable to show full chain-of-thought.
- `_ToolCallGroup` – Tool calls grouped with their results by `call_id`. Shows function name, status chip (`completed/in_progress/failed`), pretty-printed JSON arguments, and matched result – all in one expandable card.
- `_CopyIconButton` – Copy-to-clipboard button on all cards for quick data extraction.
- `_UsageBar` – Token usage statistics (input/output/total) displayed at the bottom.
- **Refusal handling** – `RefusalPart` content renders with error styling and red-tinted background.

All widgets use Material Design 3 theming and adapt to dark/light mode.



Raw Open Responses JSON — how every tool shows it today



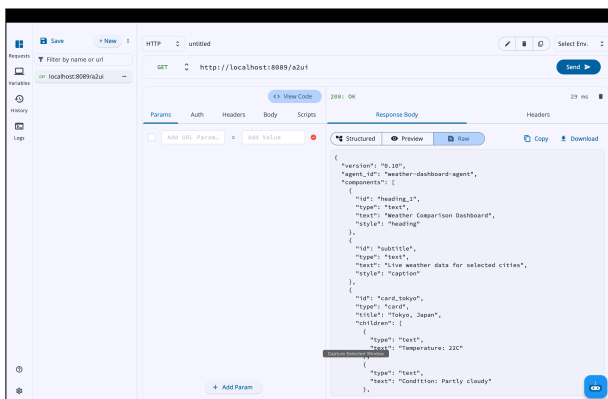
Early prototype: structured card view (reasoning, tool calls, results separated). Much more to be built during GSoC.

3. A2UI Component Renderer (lib/widgets/a2ui_renderer.dart – 349 lines)

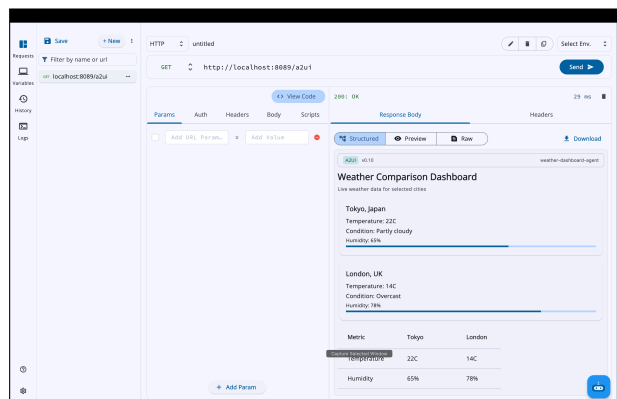
A registry-based renderer that maps **A2UI** component type strings to Flutter widget builders. The **A2UIParser** class handles detection (**isA2UIPayload**) and parsing of A2UI JSONL payloads. The **A2UIRenderer** is a **StatefulWidget** with local form state management, supporting **15 component types**:

- **Layout:** Card, Row, Column, List, Tabs
- **Display:** Text (with h1/h2/h3/caption variants), Image, Icon, Divider
- **Input:** Button (primary/outlined/text variants), TextField, Checkbox, Switch
- **Feedback:** Progress (determinate/indeterminate), Chip
- **Data Binding:** JSON Pointer path resolution from the data model (e.g., `/user/name`)

Interactive components (TextField, Checkbox, Switch) maintain local state. Buttons show snackbar with action name. Unknown component types fall back to an error card showing the type name, so the renderer degrades gracefully.



A2UI component descriptors shown as raw JSON — no tool renders this today



Early prototype: same payload as a live interactive dashboard. Full A2UI spec coverage and streaming are planned for GSoC.

4. Response Pipeline Integration

Extended the existing response pipeline with:

- Two new **ResponseBodyView** options: **structured** (for Open Responses) and **genui** (for A2UI).

- Auto-detection logic in `_resolveViewOptions()` that checks for Open Responses format (`object == "response" + output[]` array) and A2UI format (`createSurface/updateComponents` keys).
- Routing to the appropriate viewer in `ResponseBodySuccess`.

How I'll Build This – Technical Approach for GSoC

The PoC proves the concept works. But there's a meaningful gap between “it works on fixture data” and “it handles real-world streaming responses with edge cases.” Here's exactly how I plan to close that gap:

A. Streaming with Typed SSE Events

Open Responses streaming uses typed events like `response.output_item.added`, `response.output_text.delta`, `response.function_call_arguments.delta`, and `response.output_item.done`. Each event targets a specific output item by index, so the parser needs to maintain a map of in-progress items and route deltas to the right place.

```
class OpenResponsesStreamState {
    final List<OutputItem> items = [];
    final Map<int, StringBuffer> textBuffers = {};
    final Map<int, StringBuffer> argBuffers = {};

    void processEvent(String eventType,
        Map<String, dynamic> data) {
        switch (eventType) {
            case 'response.output_item.added':
                items.add(
                    OutputItem.fromJson(data['item']));
            case 'response.output_text.delta':
                textBuffers[data['output_index']]?
                    .write(data['delta']);
            case 'response.output_item.done':
                _finalizeItem(
                    data['output_index'], data['item']);
        }
    }
}
```

B. Conversation Chaining (`previous_response_id`)

Implementation plan:

- Store `response.id` from each parsed response in the request model.
- Wire the stored ID into subsequent requests as `previous_response_id`.
- Build a timeline view that links related requests, showing the full conversation flow.
- Allow expanding/collapsing individual turns in the chain.

C. Extended A2UI Component Support

The PoC handles 15 component types with interactive state. The full implementation will expand this to cover the [A2UI specification](#) more completely:

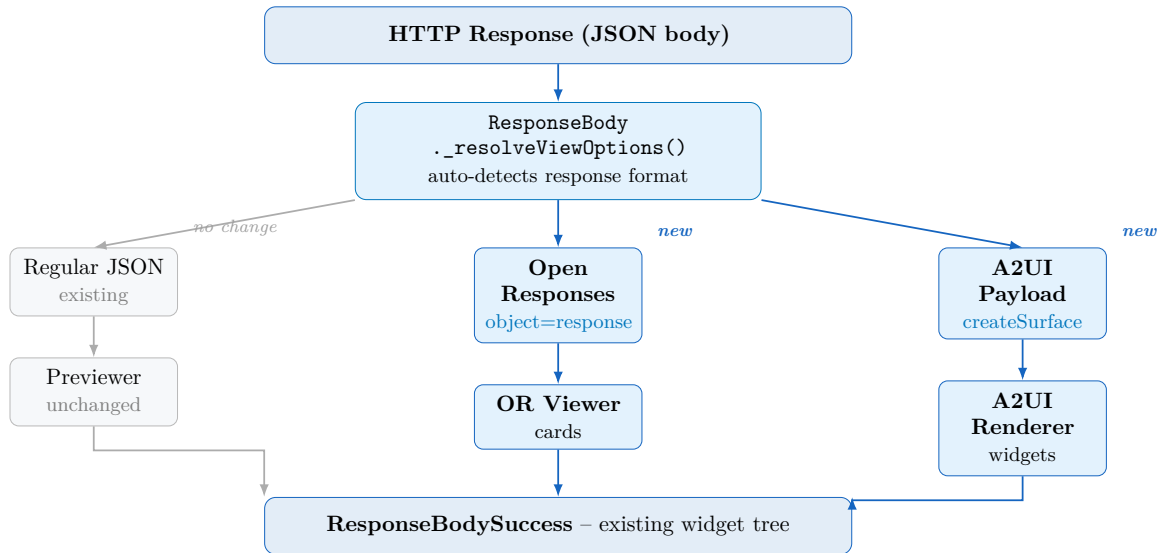
- **Input components:** TextField, DatePicker, Slider, Checkbox, Radio, Dropdown
- **Data display:** Table (with sorting/filtering), Progress bar, Charts (using `fl_chart`)
- **Navigation:** Tabs, Accordion, Stepper
- **Local state management:** Form state tracking, user interaction event logging
- **Progressive rendering:** Components render as they stream in via A2UI's flat ID-referenced model

D. DashBot Integration

Currently, DashBot renders all responses through `MarkdownBody` in `ChatBubble`. When DashBot's underlying AI provider returns an Open Responses format response, the integration would:

- Parse the response using the same `OpenResponsesResult.fromJson` pipeline.
- Render structured output items inline in the chat (reasoning collapsed, tool calls as cards).
- If the response contains [A2UI](#) components, render them as interactive widgets within the chat bubble.

Architecture Diagram



What the End Result Will Look Like

Scenario 1 – Testing an Open Responses endpoint (non-streaming):

You send a request to an endpoint that follows the [Open Responses protocol](#). Instead of seeing a JSON blob, the response pane auto-switches to a structured view. You see the model's reasoning as a collapsed purple card at the top (click to expand the full chain-of-thought). Below that, tool calls show as blue cards – each one shows the function name, a status chip, a pretty-printed version of the arguments, and the matching result right below it. The actual message text renders at the bottom as formatted markdown with inline images. A token usage bar at the bottom tells you how many tokens were used.

Scenario 2 – Streaming an agentic workflow:

You send a streaming request. Cards appear one by one as the model generates output. First, a reasoning card pops up and its summary text fills in progressively. Then a function call card appears – the function name shows immediately, and the arguments string grows character by character. When the tool result comes back, it slides in below the matching call. Finally, the message card appears and text streams into it. At no point do you see raw SSE events or JSON fragments.

Scenario 3 – Previewing an [A2UI](#) agent response:

You send a request to a Vertex AI agent that returns A2UI component descriptors. Instead of seeing the JSON describing buttons, forms, and tables, you see actual rendered Flutter widgets – interactive buttons you can tap, text fields you can type into, data tables with rows and columns. An event log panel shows what interactions the rendered UI generates, so you can verify your agent's UI behaves correctly.

Scenario 4 – Debugging a multi-turn agentic loop:

You make three chained requests (each using `previous_response_id`). API Dash links them in a conversation timeline. You can see the full flow: first request → model reasons → calls tool A → you send result → model reasons again → calls tool B → you send result → model gives final answer. All in one connected view, not scattered across separate request tabs.

What Could Go Wrong and How I'd Handle It

Risk	Impact	Mitigation
Extending <code>outputFormatter</code>	High	Add a parallel <code>structuredOutputFormatter</code> path
A2UI spec changes	Medium	Pin to specific version; unknown types show raw JSON
Streaming takes longer	Medium	Non-streaming structure works first as fallback
Multi-turn chain state	Medium	Start with in-session track, persist later
Testing without API keys	Low	Already solved – PoC includes fixture payloads

Implementation Plan

Project Size: 175 hours over 12 weeks (~15 hours/week)

Community Bonding Period (May 8 – June 1)

- Attend [weekly mentor sync-ups](#) and get feedback on the [PoC](#).
- Address any review comments on [PR #1321](#) (idea doc) and the PoC PR.
- Finalize design decisions with mentors: how to extend the provider interface for structured output, where to store conversation chain state, and the DashBot integration approach.
- Set up test fixtures for all edge cases (malformed responses, partial data, unknown output types).

Phase 1 – Open Responses Production Parser & Structured Viewer (Weeks 1–3)

Goal: Turn the PoC's parsing and rendering into production-quality code.

Week 1 (June 2 – June 8)

- Finalize the `OpenResponsesResult` data models based on mentor feedback.
- Add handling for `output_image` and `output_file` content parts (PoC only handles `output_text` and refusal).
- Implement robust format auto-detection with zero false positives on non-AI responses.
- Write unit tests for all `fromJson` factories using fixture payloads.

Deliverable: *Production-ready Open Responses parser in `packages/genai` with full test coverage.*

Week 2 (June 9 – June 15)

- Polish the Structured Response Viewer widgets (reasoning, function call, message cards).
- Add markdown rendering for message text content (using existing `flutter_markdown` integration).
- Implement inline image rendering for `output_image` parts (using existing `Image.memory`).
- Add a download button for `output_file` parts.

Deliverable: *Complete structured viewer rendering all Open Responses output types.*

Week 3 (June 16 – June 22)

- Wire the `OpenResponsesModel` provider into `kModelProvidersMap`.
- Implement `createRequest()` for the Responses API format (input + instructions).
- Implement non-streaming `outputFormatter` and `structuredOutputFormatter`.
- Integration tests: send requests via UI, verify auto-detection and structured rendering.

Deliverable: *Working end-to-end flow – user sees structured cards instead of raw JSON.*

Phase 2 – Streaming & Conversation Chaining (Weeks 4–6)

Goal: Handle real-time streaming responses and multi-turn agentic workflows.

Week 4 (June 23 – June 29)

- Build the `OpenResponsesStreamState` machine for typed SSE events.
- Handle core event types:
 - `response.output_item.added`
 - `response.output_text.delta`
 - `response.function_call_arguments.delta`
 - `response.output_item.done`
- Wire stream state into `streamOutputFormatter` for the Open Responses provider.

Deliverable: *Streaming parser that correctly routes typed deltas to output items.*

Week 5 (June 30 – July 6)

- Connect the stream state machine to the structured viewer for progressive rendering.
- Each output item card appears as `output_item.added` fires; text and arguments fill in as deltas arrive.
- Handle edge cases: out-of-order events, partial data, connection drops.
- Write tests with recorded SSE fixtures.

Deliverable: *Live streaming structured view – cards build up in real-time.*

Week 6 (July 7 – July 13)

- Implement `previous_response_id` tracking in the request model.
- Build the conversation timeline view linking related requests.
- Allow expanding/collapsing individual turns in the chain.
- Mentor review before midterm evaluation.

Deliverable: *Multi-turn conversation chaining with a connected timeline view.*

Midterm Evaluation (July 14 – July 18)

Expected state: Full Open Responses support (parsing, structured viewing, streaming, conversation chaining) working end-to-end.

Phase 3 – A2UI Component Renderer (Weeks 7–9)

Goal: Extend the PoC's [A2UI](#) renderer to cover the full spec with interactive state and event logging.

Week 7 (July 21 – July 27)

- Expand the component registry to cover all core A2UI types: `TextField`, `DatePicker`, `Slider`, `Checkbox`, `Radio`, `Dropdown`, `Table`, `Progress`, `Accordion`, `Stepper`.
- Implement data binding resolution for all input types.
- Add local form state management.

Deliverable: *A2UI renderer supporting 20+ component types with interactive state.*

Week 8 (July 28 – August 3)

- Implement event logging system: user interactions with rendered components are captured in a collapsible log panel.
- Add progressive rendering: components render as they stream in via `updateComponents`.
- Handle component updates by ID (agents can update individual components).

Deliverable: *Interactive A2UI renderer with event logging and progressive rendering.*

Week 9 (August 4 – August 10)

- Build the A2UI detection and routing pipeline (extending `_resolveViewOptions()`).
- Add support for mixed responses (Open Responses output containing A2UI component descriptors).
- Write comprehensive tests for component rendering with various payloads.
- Mentor review.

Deliverable: *Full A2UI rendering pipeline integrated into the response viewer.*

Phase 4 – DashBot Integration & Polish (Weeks 10–12)

Goal: Bring structured AI responses into DashBot and polish everything for merge.

Week 10 (August 11 – August 17)

- Extend `ChatBubble` in DashBot to render structured Open Responses output inline.
- When DashBot receives a response with reasoning/tool calls, render them as cards within the chat.
- Add A2UI component rendering within chat bubbles for interactive agent responses.

Deliverable: *DashBot with rich structured response rendering.*

Week 11 (August 18 – August 24)

- End-to-end testing with real API endpoints (OpenAI Responses API, compatible providers).
- Edge case handling: malformed responses, partial streams, mixed content types, very large responses.
- Performance profiling: ensure structured rendering does not add noticeable latency.
- Fix bugs identified during testing.

Deliverable: *Battle-tested implementation passing all edge cases.*

Week 12 (August 25 – September 1)

- Code cleanup, documentation, and final polish.

- Ensure all new code follows API Dash’s existing patterns and conventions.
- Write developer documentation for extending the component registry.
- Final mentor review and project report.

Deliverable: *Merge-ready code with documentation.*

★ **Stretch Goals (if time permits)**

- **A2UI Live Playground:** A split-pane editor where you can edit A2UI JSON on the left and see rendered Flutter widgets update on the right in real-time.
- **Visual Conversation Debugger:** An enhanced multi-turn view with collapsible branches, search/filter by output type, and export of full conversation chains as shareable reports.

Contributions to API Dash

PR	Description	Status
#1279	Input validation for empty API key and endpoint URL in AI Model Selector dialog	Open
#1290	Custom OpenAI-compatible LLM provider support (Groq, OpenRouter, Mistral, etc.)	Open
#1321	GSoC 2026 idea doc – Open Responses & Generative UI	Merged
#1358	PoC – Open Responses structured viewer + A2UI component renderer (demo 1 , demo 2)	Open

Contributions to Other Open-Source Projects

astropy – Python, 500k+ lines, widely used astronomy library

PR	Description	Status
#19403	Fixed missing f-string prefix in <code>BaseAffineTransform._apply_transform</code>	Merged
#19404	Fixed silent data corruption in <code>FITS_rec</code> slice assignment with negative indices	Merged
#19450	Fixed table index corruption when a row assignment fails	Merged

Hyperledger Besu – Java, Ethereum execution client

PR	Description	Status
#10020	Fixed critical bug where <code>serializeAndDedupOperation()</code> never returned <code>failedFuture</code>	Merged
#10042	Fixed <code>eth_estimateGas</code> using parent block's hash instead of current block's hash	Merged
#10051	Fixed TOCTOU race condition in <code>eth_getBlockByNumber</code>	Merged
#10060	Fixed unsigned underflow in <code>eth_feeHistory</code> reward bounds calculation	Merged

Hyperledger Fabric – Go, enterprise blockchain platform

PR	Description	Status
#5422	Fixed gossip protocol not returning after sending NACK on private data store failure	Merged
#5424	Fixed resource leak: <code>collItr</code> not being closed before releasing <code>purgerLock</code>	Merged
#5430	Fixed iterator leak in <code>deleteDataMarkedForPurge</code>	Merged

LFDT-Lockness/generic-ec – Rust, elliptic curve cryptography

PR	Description	Status
#59	Added secp384r1 (NIST P-384) curve support – first non-32-byte scalar curve, 180 tests passing	Merged

Why Me

5 open-source projects · 11 merged PRs · 4 API Dash PRs · ~1,100 lines of working PoC

I'll be honest – I'm a first-year student applying to work on something that matters to me, not just something that looks good on a resume. I've been genuinely frustrated with how every API tool handles AI responses. I tested Postman, Insomnia, Thunder Client with real agentic APIs – and every single one just dumps the JSON. I kept thinking *“there has to be a better way to see this.”* That frustration is what started all of this.

Before I wrote a single line of proposal text, I spent two weeks just reading the API Dash codebase. Not skimming – actually reading. I traced how a raw HTTP response becomes a rendered widget. I read how `ModelProvider` abstracts different AI backends, how `ResponseBodyView` decides what to show, how the Stac pipeline connects provider output to the UI. It took a while, and I got lost more than once. But by the end of it I knew exactly where my code had to live – not just where it would work, but where it *belonged*.

Then I built the PoC. Not because the application required it, but because I genuinely wasn't sure I could do it – and I needed to find out. The sealed class hierarchy, the widget tree, the A2UI registry, wiring it all into the existing pipeline without breaking anything – **1,100 lines** later, it works. There are still hard parts ahead (the streaming state machine, conversation chaining), but I now know the codebase well enough to know where those pieces go.

What I want to build isn't just a feature. It's something I'd use every day, and something every developer testing AI APIs would benefit from. That's not a pitch – it's genuinely why I chose this project over everything else in the ideas list.